```
# ==========================================
# 1. IMPORT REQUIRED LIBRARIES
# ==========================================

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.decomposition import PCA

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    r2_score
)

from xgboost import XGBRegressor

import warnings
warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
train_df = pd.read_csv("train.csv")
test_df = pd.read_csv("test.csv")

print("Train Shape:", train_df.shape)
print("Test Shape:", test_df.shape)
```

```
Train Shape: (4209, 378)
Test Shape: (4209, 377)
```

```
print("First 5 rows of training data:")
display(train_df.head())

print("Last 5 rows:")
display(train_df.tail())

print("Training Dataset Information:")
train_df.info()

print("Test Dataset Information:")
test_df.info()
```

```
display(train_df.describe().T)
```

```
print("Missing values in TRAIN:")
print(train_df.isnull().sum().sum())

print("\nMissing values in TEST:")
print(test_df.isnull().sum().sum())
```

```
Missing values in TRAIN:
0

Missing values in TEST:
0
```

```
train_missing = train_df.isnull().sum()
train_missing = train_missing[train_missing > 0]

test_missing = test_df.isnull().sum()
test_missing = test_missing[test_missing > 0]

print("Train columns with missing values:")
print(train_missing)

print("\nTest columns with missing values:")
print(test_missing)
```

```
Train columns with missing values:
Series([], dtype: int64)

Test columns with missing values:
Series([], dtype: int64)
```

```
print("Duplicate rows in train:", train_df.duplicated().sum())
print("Duplicate rows in test:", test_df.duplicated().sum())
```

```
Duplicate rows in train: 0
Duplicate rows in test: 0
```

```
unique_counts = train_df.nunique().sort_values()

display(unique_counts.head(30))
```

```
categorical_cols = [
    'X0',
    'X1',
    'X2',
    'X3',
    'X4',
    'X5',
    'X6',
    'X8'
]

print("Categorical columns:")
print(categorical_cols)
```

```
Categorical columns:
['X0', 'X1', 'X2', 'X3', 'X4', 'X5', 'X6', 'X8']
```

```
binary_cols = []

for col in train_df.columns:
    if col not in categorical_cols and col not in ['ID', 'y']:
        if train_df[col].nunique() <= 2:
            binary_cols.append(col)
```

```
print("Number of binary columns:", len(binary_cols))
print("\nFirst 20 binary columns:")
print(binary_cols[:20])
```

```
Number of binary columns: 368

First 20 binary columns:
['X10', 'X11', 'X12', 'X13', 'X14', 'X15', 'X16', 'X17', 'X18', 'X19', 'X20', 'X21', 'X22', '
```

```
zero_variance_cols = []

for col in train_df.columns:
    if train_df[col].nunique() == 1:
        zero_variance_cols.append(col)

print("Number of zero-variance columns:",
      len(zero_variance_cols))

print("\nZero-variance columns:")
print(zero_variance_cols)
```

```
Number of zero-variance columns: 12

Zero-variance columns:
['X11', 'X93', 'X107', 'X233', 'X235', 'X268', 'X289', 'X290', 'X293', 'X297', 'X330', 'X347'
```

```
train_df = train_df.drop(columns=zero_variance_cols)
test_df = test_df.drop(columns=zero_variance_cols)

print("Train shape after removing zero-variance columns:",
      train_df.shape)

print("Test shape after removing zero-variance columns:",
      test_df.shape)
```

```
Train shape after removing zero-variance columns: (4209, 366)
Test shape after removing zero-variance columns: (4209, 365)
```

```
# Target variable
y = train_df['y']

# Remove target from training data
X_train = train_df.drop(columns=['y'])

# Remove ID because it is only an identifier
X_train = X_train.drop(columns=['ID'])
X_test = test_df.drop(columns=['ID'])

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y shape:", y.shape)
```

```
X_train shape: (4209, 364)
X_test shape: (4209, 364)
y shape: (4209,)
```

```
# ============================================
# 12. TARGET VARIABLE ANALYSIS
# ============================================
```

```
print(y.describe())
```

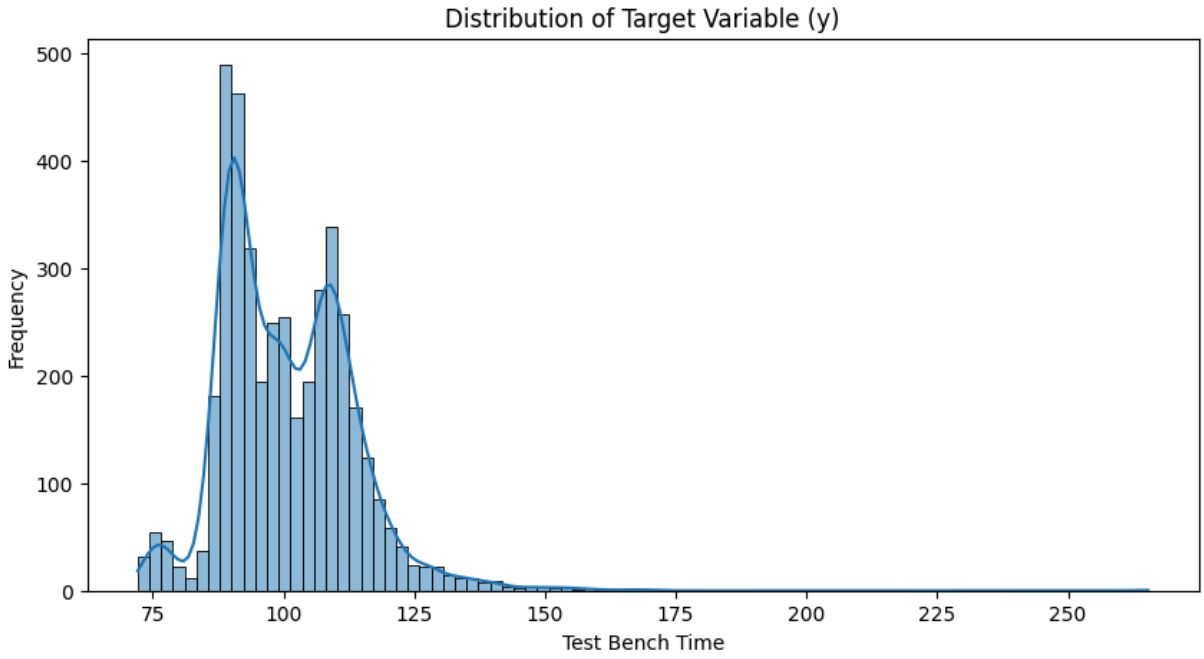
```
count    4209.000000
mean      100.669318
std        12.679381
min        72.110000
25%        90.820000
50%        99.150000
75%       109.010000
max       265.320000
Name: y, dtype: float64
```

```
plt.figure(figsize=(10, 5))

sns.histplot(y, kde=True)

plt.title("Distribution of Target Variable (y)")
plt.xlabel("Test Bench Time")
plt.ylabel("Frequency")

plt.show()
```

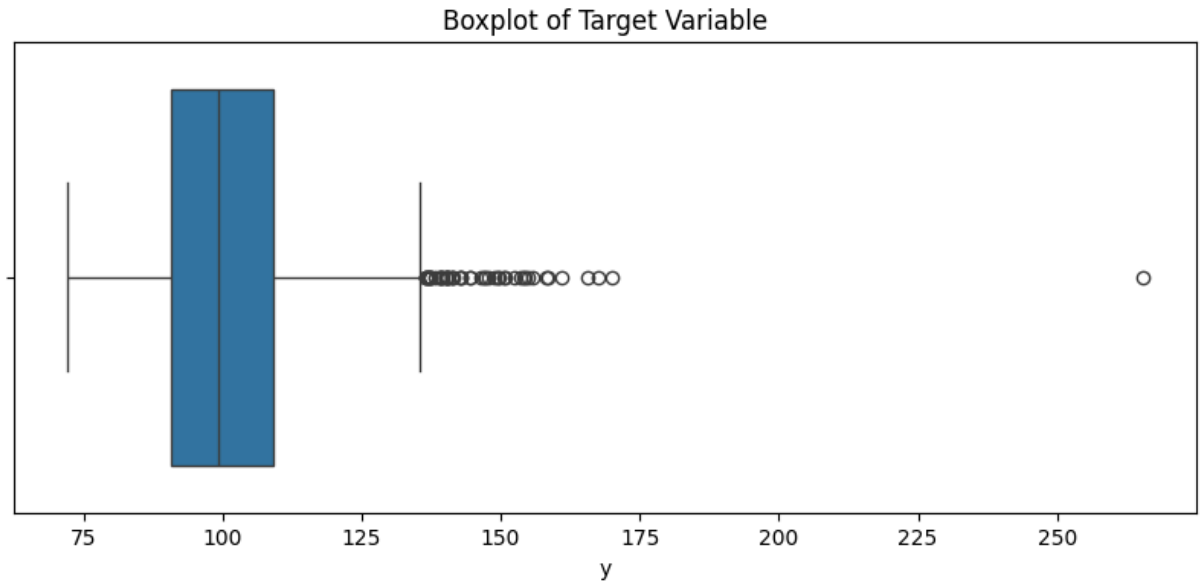



```
plt.figure(figsize=(10, 4))

sns.boxplot(x=y)

plt.title("Boxplot of Target Variable")

plt.show()
```




```
# ==========================================
# 13. LABEL ENCODING
# ==========================================

label_encoders = {}

for col in categorical_cols:

    le = LabelEncoder()

    # Combine train and test values
    combined_values = pd.concat(
        [X_train[col], X_test[col]],
        axis=0
    ).astype(str)

    # Fit encoder on combined categories
    le.fit(combined_values)

    # Transform train and test
    X_train[col] = le.transform(X_train[col].astype(str))
    X_test[col] = le.transform(X_test[col].astype(str))

    # Store encoder
    label_encoders[col] = le

print("Label encoding completed successfully!")
```

```
Label encoding completed successfully!
```

```
display(X_train.head())
```

```
print(X_train.dtypes.value_counts())
```

```
int64    364
Name: count, dtype: int64
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
print("Scaling completed!")
```

```
Scaling completed!
```

```
# ==========================================
# 16. PCA - EXPLAINED VARIANCE
# ==========================================

pca_full = PCA()

X_train_pca_full = pca_full.fit_transform(X_train_scaled)

explained_variance = pca_full.explained_variance_ratio_

cumulative_variance = np.cumsum(explained_variance)

plt.figure(figsize=(10, 6))

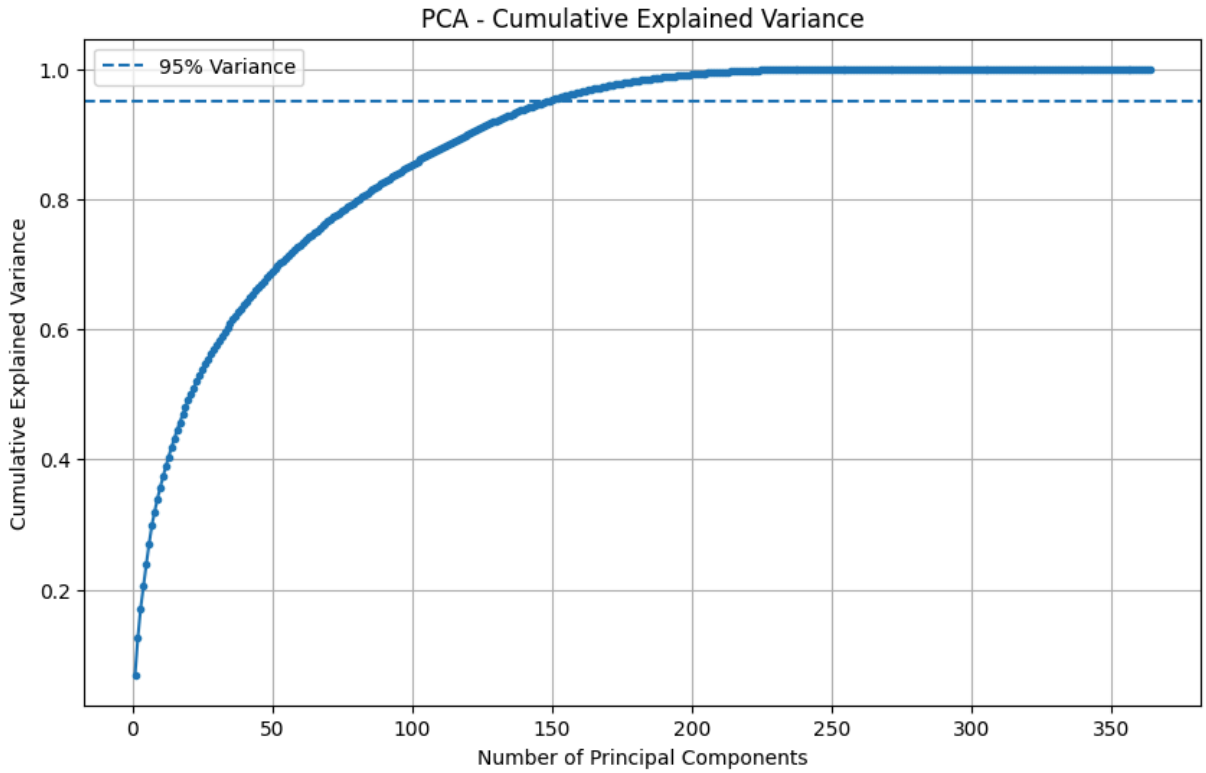
plt.plot(
    range(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker='.'
)

plt.axhline(
    y=0.95,
    linestyle='--',
    label='95% Variance'
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("PCA - Cumulative Explained Variance")

plt.legend()
plt.grid()

plt.show()
```




```
# ============================================
# 17. SELECT PCA COMPONENTS
# ============================================

n_components = np.argmax(
    cumulative_variance >= 0.95
) + 1

print("Number of components required for 95% variance:",
      n_components)
```

```
Number of components required for 95% variance: 148
```

```
pca = PCA(n_components=n_components)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Original number of features:",
      X_train.shape[1])

print("Reduced number of features:",
      X_train_pca.shape[1])
```

```
Original number of features: 364
Reduced number of features: 148
```

```
# ============================================
# 18. TRAIN-VALIDATION SPLIT
# ============================================

X_train_split, X_val, y_train, y_val = train_test_split(
    X_train_pca,
    y,
    test_size=0.20,
```

```
    random_state=42
)

print("Training data:", X_train_split.shape)
print("Validation data:", X_val.shape)
```

```
Training data: (3367, 148)
Validation data: (842, 148)
```

```
# ==========================================
# 19. XGBOOST REGRESSION MODEL
# ==========================================

xgb_model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=5,
    subsample=0.8,
    colsample_bytree=0.8,
    objective='reg:squarederror',
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(
    X_train_split,
    y_train
)

print("XGBoost model trained successfully!")
```

```
XGBoost model trained successfully!
```

```
y_pred = xgb_model.predict(X_val)

print("First 10 predictions:")
print(y_pred[:10])
```

```
First 10 predictions:
[106.68713   96.04857  107.6098    77.178734 106.70466   97.47298
  94.36018  101.04731  101.829956 110.16292 ]
```

```
# ==========================================
# 21. MODEL EVALUATION
# ==========================================

mae = mean_absolute_error(y_val, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_val, y_pred)
)

r2 = r2_score(y_val, y_pred)

print("Model Performance")
print("-------------------------")
print("MAE  :", mae)
print("RMSE :", rmse)
print("R²   :", r2)
```

```
Model Performance
-------------------------
MAE  : 6.422382770456782
```

```
RMSE : 9.369853599993446
R²   : 0.43595123335191777
```

```
# ==========================================
# 22. ACTUAL VS PREDICTED
# ==========================================

plt.figure(figsize=(8, 6))

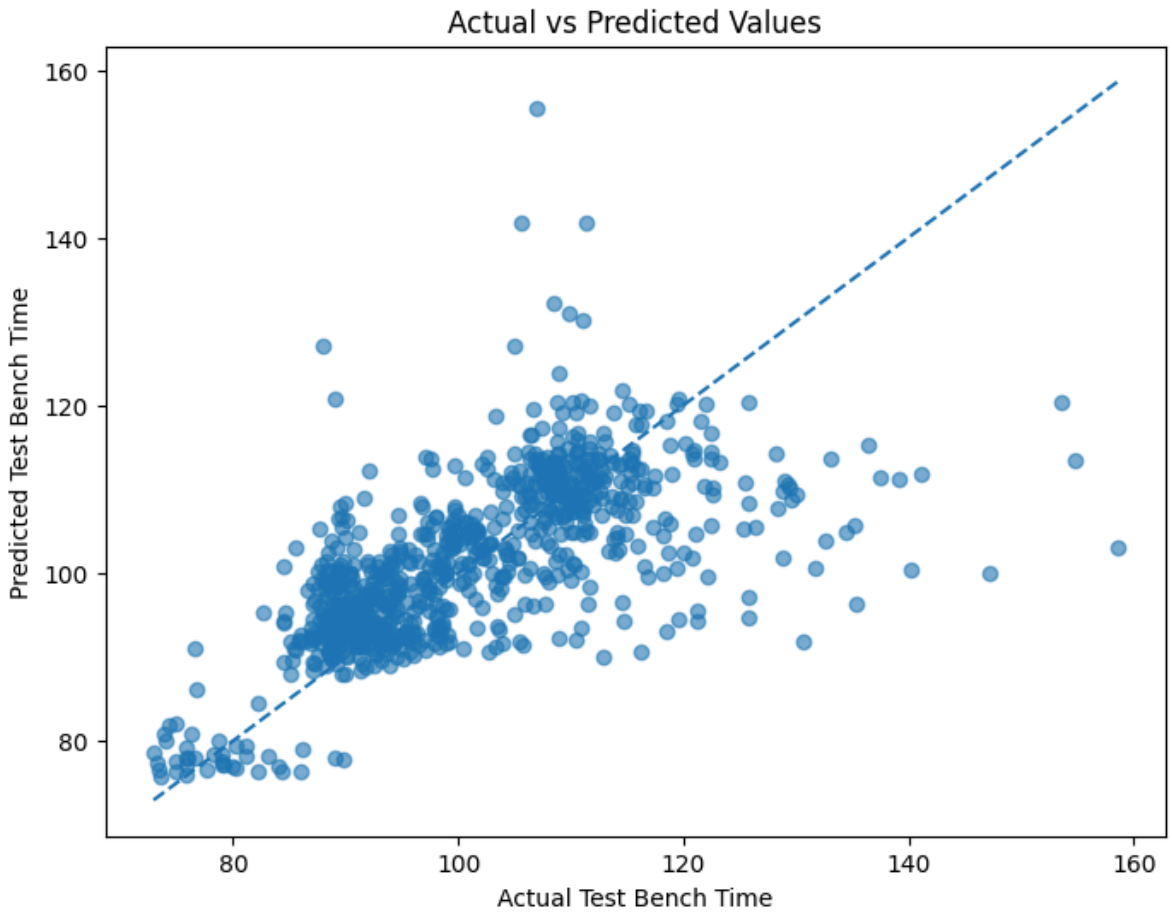
plt.scatter(
    y_val,
    y_pred,
    alpha=0.6
)

plt.xlabel("Actual Test Bench Time")
plt.ylabel("Predicted Test Bench Time")

plt.title("Actual vs Predicted Values")

plt.plot(
    [y_val.min(), y_val.max()],
    [y_val.min(), y_val.max()],
    linestyle='--'
)

plt.show()
```



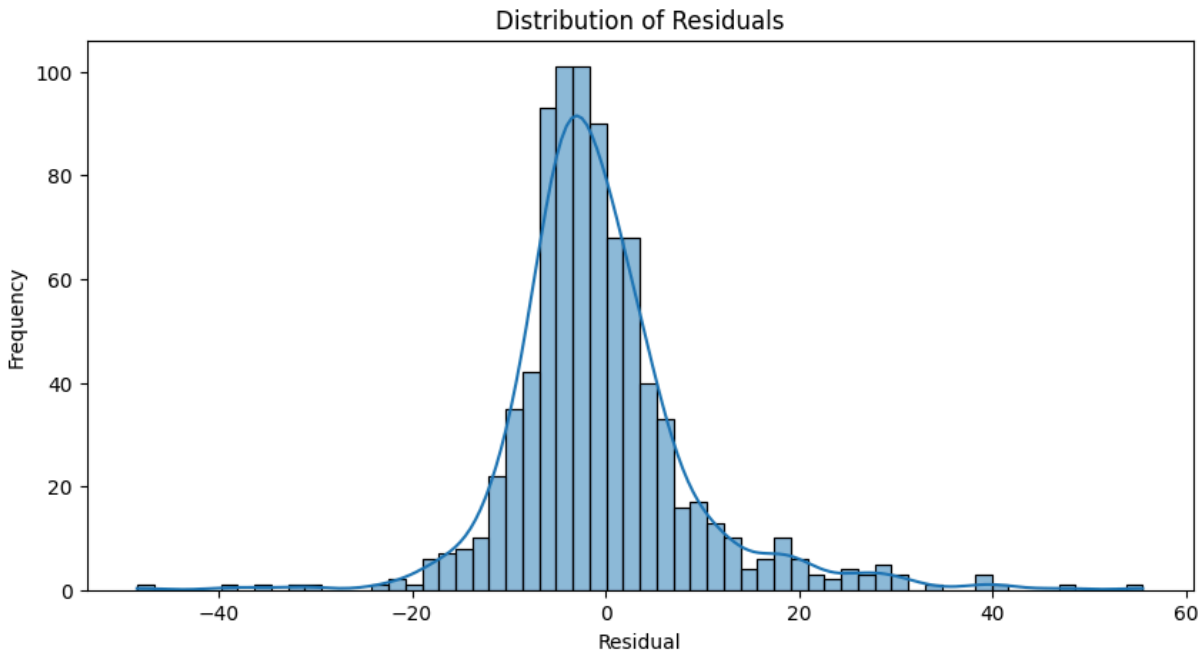

```
residuals = y_val - y_pred

plt.figure(figsize=(10, 5))
```

```
sns.histplot(residuals, kde=True)

plt.title("Distribution of Residuals")
plt.xlabel("Residual")
plt.ylabel("Frequency")

plt.show()
```




```
# ==========================================
# 24. HYPERPARAMETER TUNING
# ==========================================

xgb = XGBRegressor(
    objective='reg:squarederror',
    random_state=42,
    n_jobs=-1
)

param_grid = {
    'n_estimators': [200, 400],
    'max_depth': [3, 5],
    'learning_rate': [0.03, 0.05],
    'subsample': [0.8],
    'colsample_bytree': [0.8]
}

grid_search = GridSearchCV(
    estimator=xgb,
    param_grid=param_grid,
    cv=3,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(
    X_train_split,
    y_train
)
```

```
Fitting 3 folds for each of 8 candidates, totalling 24 fits
```

▸ **GridSearchCV**

▸ **best_estimator_: XGBRegressor**

▸ XGBRegressor

```
print("Best Parameters:")
print(grid_search.best_params_)

print("\nBest CV RMSE:")
print(-grid_search.best_score_)
```

```
Best Parameters:
{'colsample_bytree': 0.8, 'learning_rate': 0.03, 'max_depth': 3, 'n_estimators': 400, 'subsam

Best CV RMSE:
9.137992801475919
```

```
# ============================================
# 25. FINAL MODEL
# ============================================

best_model = grid_search.best_estimator_

best_model.fit(
    X_train_pca,
    y
)

print("Final model trained successfully!")
```

```
Final model trained successfully!
```

```
# ============================================
# 26. TEST DATA PREDICTION
# ============================================

test_predictions = best_model.predict(
    X_test_pca
)

print("Number of predictions:",
      len(test_predictions))

print("\nFirst 10 predictions:")
print(test_predictions[:10])
```

```
Number of predictions: 4209

First 10 predictions:
[105.2798   123.2277   104.367775  81.93632  103.95083   93.40536
 105.5446    94.61524  116.591515  96.37004 ]
```

```
# ============================================
# 27. CREATE SUBMISSION FILE
# ============================================

submission = pd.DataFrame({
    'ID': test_df['ID'],
```

```
    'y': test_predictions
})

display(submission.head())
```

	ID	y
0	1	105.279800
1	2	123.227699
2	3	104.367775
3	4	81.936317
4	5	103.950829

```
submission.to_csv(
    "Mercedes_Benz_predictions.csv",
    index=False
)

print("Prediction file created successfully!")
```

```
Prediction file created successfully!
```

```
# ============================================
# 28. VERIFY SUBMISSION
# ============================================

final_submission = pd.read_csv(
    "Mercedes_Benz_predictions.csv"
)

print(final_submission.shape)

display(final_submission.head())
```